

Session 3



Intermediate R

Conditionals and Control Flow

Equality

The most basic form of comparison is equality. Let's briefly recap its syntax. The following statements all evaluate to `TRUE`.

```
3 == (2 + 1)
"intermediate" != "r"
TRUE != FALSE
"Rchitect" != "rchitect"
```

Notice from the last expression that R is case sensitive: "R" is not equal to "r". Keep this in mind when solving the exercises in this chapter!

Greater and less than

Apart from equality operators, also introduced the less than and greater than operators: `<` and `>`. You can also add an equal sign to express less than or equal to or greater than or equal to, respectively. Have a look at the following R expressions, that all evaluate to `FALSE`:

```
(1 + 2) > 4
```

```
"dog" < "Cats"
```

```
TRUE <= FALSE
```

Remember that for string comparison, R determines the greater than relationship based on alphabetical order. Also, keep in mind that `TRUE` is treated as 1 for arithmetic, and `FALSE` is treated as 0. Therefore, `FALSE < TRUE` is `TRUE`.

Compare vectors

You are already aware that R is very good with vectors. Without having to change anything about the syntax, R's relational operators also work on vectors.

Compare matrices

R's ability to deal with different data structures for comparisons does not stop at vectors. Matrices and relational operators also work together seamlessly!

Logical Operators

& and |

Before you work your way through the next exercises, have a look at the following R expressions. All of them will evaluate to *TRUE*:

```
TRUE & TRUE
```

```
FALSE | TRUE
```

```
5 <= 5 & 2 < 3
```

```
3 < 4 | 7 < 6
```

Watch out: `3 < x < 7` to check if `x` is between 3 and 7 will not work; you'll need `3 < x & x < 7` for that.

Conditional Statements

The `if` statement

Before diving into some exercises on the `if` statement, have another look at its syntax:

```
if (condition) {  
    expr  
}
```

Add an else

You can only use an else statement in combination with an if statement. The else statement does not require a condition; its corresponding code is simply run if all of the preceding conditions in the control structure are FALSE. Here's a recipe for its usage:

```
if (condition) {  
    expr1  
} else {  
    expr2  
}
```

Customize further: else if

The else if statement allows you to further customize your control structure. You can add as many else if statements as you like. Keep in mind that R ignores the remainder of the control structure once a condition has been found that is TRUE and the corresponding expressions have been executed. Here's an overview of the syntax to freshen your memory:

```
if (condition1) {  
  expr1  
} else if (condition2) {  
  expr2  
} else {  
  expr4  
}
```

Else if 2.0

You can do anything you want inside if-else constructs. You can even put in another set of conditional statements. Examine the following code chunk:

```
if (number < 10) {  
  if (number < 5) {  
    result <- "extra small"  
  } else {  
    result <- "small"  
  }  
} else if (number < 100) {  
  result <- "medium"  
} else {  
  result <- "large"  
}  
print(result)
```

Loops

Write a while loop

Let's get you started with building a while loop from the ground up. Have another look at its recipe:

```
while (condition) {  
    expr  
}
```

Remember that the condition part of this recipe should become *FALSE* at some point during the execution. Otherwise, the while loop will go on indefinitely.

Stop the while loop: break

The `break` statement is a control statement. When R encounters it, the while loop is abandoned completely.

Build a while loop from scratch

The previous exercises guided you through developing a pretty advanced while loop, containing a break statement and different messages and updates as determined by control flow constructs.



Break
10 minutes

For loop

```
for (val in sequence)
{
  statement
}
```

Here, sequence is a vector and val takes on each of its value during the loop. In each iteration, statement is evaluated.

Loop over a vector

Loop over a list

Loop over a matrix

Mix it up with control flow

Time to throw in some conditionals! As with the while loop, you can use the if and else statements inside the for loop.